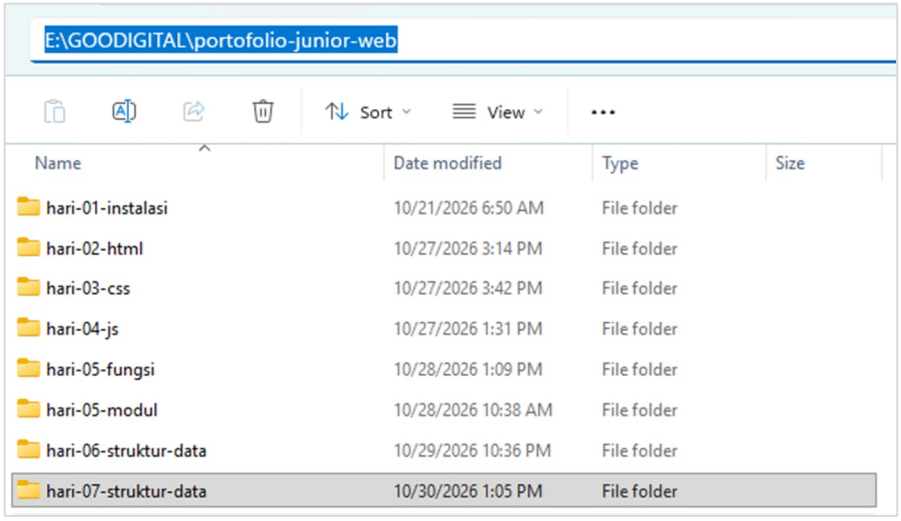
Modul Hari 7 - Struktur Data II, Algoritma, dan Clean Code (Proyek 3)

Lab 1 - Linked List

1. LinkedList dan LinkedQueue

- Membuat folder hari-07-struktur-data.
- Membuat file structures.js.
- Menambahkan class Stack dan Queue
- Menambahkan class Node.
- Menambahkan class LinkedList.
- Menambahkan class LinkedQueue.



hari-07-struktur-data > JS structures.js > LinkedQueue

```
1  export class Stack {
2
3    constructor() {
4      this.items = [];
5    }
6
7    push(item) {
8      this.items.push(item);
9    }
10
11   pop() {
12     return this.isEmpty()
13       ? undefined
14       : this.items.pop();
15   }
16
```

hari-07-struktur-data > JS structures.js > LinkedQueue

```
37  export class Queue {
38
39    constructor() {
40      this.items = {};
41      this.head = 0;
42      this.tail = 0;
43    }
44
45    enqueue(item) {
46      this.items[this.tail] = item;
47      this.tail++;
48    }
49
```

```
hari-07-struktur-data > JS structures.js > LinkedQueue
79  class Node {
80
81    constructor(data) {
82      this.data = data;
83      this.next = null;
84    }
85
86  }
87
88  export class LinkedList {
89
90    #head = null;
91    #tail = null;
92    #size = 0;
93
94    addFirst(data) {
95
96      const node = new Node(data);
97
98      node.next = this.#head;
99
100     this.#head = node;
101
102     if (this.#tail === null) {
103       this.#tail = node;
104     }
```

```
193 export class LinkedQueue {
194
195   #daftar = new LinkedList();
196
197   enqueue(item) {
198     this.#daftar.addLast(item);
199   }
200
201   dequeue() {
202     return this.#daftar.removeFirst();
203   }
```

2. Uji LinkedList (addFirst, addLast, removeFirst, peekFirst)

- Membuat file lab1.js.
- Menguji operasi addFirst(), addLast(), removeFirst(), dan peekFirst().

```
hari-07-struktur-data > JS lab1.js > ...
1  import { LinkedList } from './structures.js';
2
3  const daftar = new LinkedList();
4
5  daftar.addFirst('B');
6  daftar.addFirst('A');
7
8  daftar.addLast('C');
9  daftar.addLast('D');
10
11 console.log('Isi LinkedList');
12 console.log(daftar.toArray());
13
14 console.log('\nPeek First');
15 console.log(daftar.peekFirst());
16
17 console.log('\nRemove First');
18 console.log(daftar.removeFirst());
19
20 console.log('\nIsi Setelah Remove');
21 console.log(daftar.toArray());
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1.js
Isi LinkedList
[ 'A', 'B', 'C', 'D' ]

Peek First
A

Remove First
A

Isi Setelah Remove
[ 'B', 'C', 'D' ]
```

3. Data pada LinkedList (find)

- Menggunakan method find().
- Mencari data berdasarkan kondisi tertentu.

```
hari-07-struktur-data > JS lab1.js > ...
14  console.log('\nPeek First');
15  console.log(daftar.peekFirst());
16
17  console.log('\nRemove First');
18  console.log(daftar.removeFirst());
19
20  console.log('\nIsi Setelah Remove');
21  console.log(daftar.toArray());
22
23  console.log('\nCari Data');
24
25  console.log(
26    daftar.find((item) => item === 'C')
27  );
28
29  console.log(
30    daftar.find((item) => item === 'Z')
31  );
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1.js
Isi LinkedList
[ 'A', 'B', 'C', 'D' ]

Peek First
A

Remove First
A

Isi Setelah Remove
[ 'B', 'C', 'D' ]

Cari Data
C
undefined
```

4. Uji LinkedQueue (enqueue, dequeue, front)

- Menggunakan class LinkedQueue.
- Menambahkan data ke antrean (enqueue).
- Mengambil data dari antrean (dequeue).
- Melihat data terdepan (front).

```
hari-07-struktur-data > JS lab1.js > ...
36  const antrean = new LinkedQueue();
37
38  antrean.enqueue('A');
39  antrean.enqueue('B');
40  antrean.enqueue('C');
41
42  console.log('\nFront');
43  console.log(
44    antrean.front()
45  );
46
47  console.log('\nDequeue 1');
48  console.log(
49    antrean.dequeue()
50  );
51
52  console.log('\nDequeue 2');
53  console.log(
54    antrean.dequeue()
55  );
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1.js
Front
A

Dequeue 1
A

Dequeue 2
B

Isi Antrean
[ 'C' ]
```

5. Uji Kasus Kosong dan Satu Elemen

- Menguji LinkedList dalam kondisi kosong.
- Menguji LinkedList dengan satu elemen.
- Mengamati hasil operasi dasar pada kondisi khusus.

```
hari-07-struktur-data > JS lab1.js > ...
62  const kosong = new LinkedList();
63
64  console.log('\nLinkedList Kosong');
65  console.log(kosong.toArray());
66  console.log(kosong.peekFirst());
67  console.log(kosong.removeFirst());
68
69  const satuData = new LinkedList();
70
71  satuData.addFirst('Satu');
72
73  console.log('\nLinkedList Satu Elemen');
74  console.log(satuData.toArray());
75  console.log(satuData.peekFirst());
76
77  console.log('\nRemove Satu Elemen');
78  console.log(satuData.removeFirst());
79
80  console.log('\nSetelah Remove');
81  console.log(satuData.toArray());
82
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1.js
LinkedList Kosong
[]
undefined
undefined

LinkedList Satu Elemen
[ 'Satu' ]
Satu

Remove Satu Elemen
Satu

Setelah Remove
[]
```

6. Hapus Node pada LinkedList (remove)

- Membuat sketsa perubahan pointer pada LinkedList.
- Menambahkan method remove(data).
- Menguji penghapusan node pada posisi kepala, tengah, ekor, dan data yang tidak ditemukan.

Perubahan Pointer pada Linked List

Kasus 1 - Hapus Node Kepala

Sebelum

[A] -> [B] -> [C]

Sesudah

[B] -> [C]

Kasus 2 - Hapus Node Tengah

Sebelum

[A] -> [B] -> [C]

Sesudah

[A] -> [C]

Kasus 3 - Hapus Node Ekor

Sebelum

[A] -> [B] -> [C]

Sesudah

[A] -> [B]

Kasus 4 - Data Tidak Ditemukan

Sebelum

[A] -> [B] -> [C]

Hapus: Z

Sesudah

[A] -> [B] -> [C]

hari-07-struktur-data > JS structures.js > ...

```
191   remove(data) {
192
193     if (this.#head === null) {
194       return false;
195     }
196
197     if (this.#head.data === data) {
198       this.removeFirst();
199       return true;
200     }
201
202     let saatIni = this.#head;
203
204     while (
205       saatIni.next !== null &&
206       saatIni.next.data !== data
207     ) {
208       saatIni = saatIni.next;
209     }
210
```

```
83  const hapusData = new LinkedList();
84
85  hapusData.addLast('A');
86  hapusData.addLast('B');
87  hapusData.addLast('C');
88
89  console.log('\nData Awal');
90  console.log(hapusData.toArray());
91
92  hapusData.remove('A');
93
94  console.log('\nHapus Kepala');
95  console.log(hapusData.toArray());
96
97  hapusData.remove('B');
98
99  console.log('\nHapus Tengah');
100 console.log(hapusData.toArray());
101
102 hapusData.addLast('D');
103 hapusData.addLast('E');
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1.js
Data Awal
[ 'A', 'B', 'C' ]

Hapus Kepala
[ 'B', 'C' ]

Hapus Tengah
[ 'C' ]

Tambah Lagi
[ 'C', 'D', 'E' ]

Hapus Ekor
[ 'C', 'D' ]

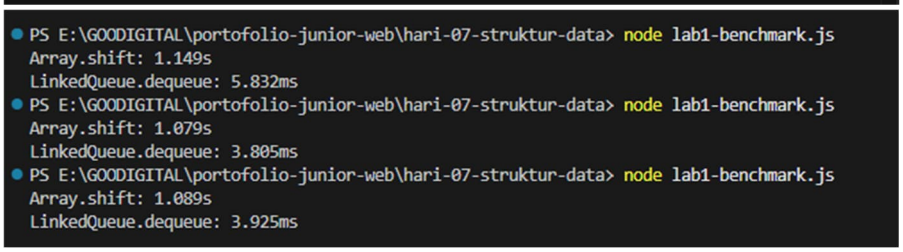
Hapus Data Tidak Ada
false
[ 'C', 'D' ]
```

7. Benchmark Queue Array vs LinkedQueue

- Membuat file lab1-benchmark.js.
- Mengukur Array.shift().
- Mengukur LinkedQueue.dequeue().
- Menjalankan pengujian 3 kali.
- Mencatat hasil pengukuran.



```
hari-07-struktur-data > JS lab1-benchmark.js > ...
1   import { LinkedQueue } from './structures.js';
2
3   const N = 100000;
4
5   const arr = [];
6
7   for (let i = 0; i < N; i++) {
8     arr.push(i);
9   }
10
11  console.time('Array.shift');
12
13  while (arr.length > 0) {
14    arr.shift();
15  }
16
17  console.timeEnd('Array.shift');
18
19  const q = new LinkedQueue();
20
21  for (let i = 0; i < N; i++) {
22    q.enqueue(i);
23  }
```



```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1-benchmark.js
Array.shift: 1.149s
LinkedQueue.dequeue: 5.832ms
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1-benchmark.js
Array.shift: 1.079s
LinkedQueue.dequeue: 3.805ms
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab1-benchmark.js
Array.shift: 1.089s
LinkedQueue.dequeue: 3.925ms
```

Pengujian	Array.shift	LinkedQueue.dequeue
1	1.149 s	5.832 ms
2	1.079 s	3.805 ms
3	1.089 s	3.925 ms
Rata-rata	1105.67 ms	4.521 ms

Lab 2 - Tree dan Graph

1. Tree Folder dan Pencarian Node

- Membuat file lab2.js.
- Menambahkan struktur data Tree.
- Menjalankan fungsi cetakPohon() dan hitungNode().

```
hari-07-struktur-data > JS lab2.js > ...
1   const folder = {
2     nama: 'proyek',
3     anak: [
4       {
5         nama: 'css',
6         anak: [
7           {
8             nama: 'style.css',
9             anak: []
10          }
11        ]
12      },
13      {
14        nama: 'js',
15        anak: [
16          {
17            nama: 'app.js',
18            anak: []
19          },
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab2.js
Struktur Folder
proyek
  css
    style.css
  js
    app.js
    api.js

Jumlah Node
6
```

2. Cari Node dengan cariNode()

- Membuat fungsi cariNode().
- Mencari node berdasarkan nama.
- Menampilkan jalur lengkap node.

```
hari-07-struktur-data > JS lab2.js > ...
52  function cariNode(node, nama, jalur = '') {
53
54    const jalurSekarang =
55      jalur
56        ? `${jalur}/${node.nama}`
57        : node.nama;
58
59    if (node.nama === nama) {
60      return jalurSekarang;
61    }
62
63    for (const anak of node.anak) {
64
65      const hasil =
66        cariNode(
67          anak,
68          nama,
69          jalurSekarang
70        );
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab2.js
Cari Node
proyek/js/app.js
```

3. Binary Search Tree (BST)

- Menambahkan class BstNode.
- Menambahkan class BinarySearchTree.
- Menambahkan data ke BST.
- Menguji inOrder().
- Menguji has().

hari-07-struktur-data > JS lab2.js > ...

```
92  class BstNode {
93
94    constructor(nilai) {
95      this.nilai = nilai;
96      this.kiri = null;
97      this.kanan = null;
98    }
99
100 }
101
102 class BinarySearchTree {
103
104   #akar = null;
105
106   insert(nilai) {
107
108     const baru = new BstNode(nilai);
109
110     if (this.#akar === null) {
111       this.#akar = baru;
112       return;
113     }
```

```
const bst = new BinarySearchTree();

bst.insert(50);
bst.insert(30);
bst.insert(70);
bst.insert(20);
bst.insert(40);
bst.insert(60);
bst.insert(80);

console.log('\nIn Order');
console.log(bst.inOrder());

console.log('\nHas 60');
console.log(bst.has(60));

console.log('\nHas 65');
console.log(bst.has(65));
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab2.js
In Order
[
  20, 30, 40, 50,
  60, 70, 80
]

Has 60
true

Has 65
false
```



4. Graf dan BFS / DFS

- Membuat graf 6 kota menggunakan adjacency list.
- Membuat fungsi BFS.
- Membuat fungsi DFS.
- Menguji penelusuran graf.

```
hari-07-struktur-data > JS lab2.js > ...
213  function bfs(graf, mulai) {
214
215    const dikunjungi = new Set();
216    const antrean = [mulai];
217
218    dikunjungi.add(mulai);
219
220    while (antrean.length > 0) {
221
222      const kota = antrean.shift();
223
```

```
241  function dfs(
242    graf,
243    mulai,
244    dikunjungi = new Set()
245  ) {
246
247    console.log(mulai);
248
249    dikunjungi.add(mulai);
250
251    for (const tetangga of graf[mulai]) {
252
253      if (!dikunjungi.has(tetangga)) {
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab2.js
BFS
Solo
Jogja
Semarang
Surabaya
Bandung
Jakarta

DFS
Solo
Jogja
Surabaya
Jakarta
Bandung
Semarang
```

5. Fungsi Jarak Terpendek (BFS)

- Membuat fungsi pencari jarak terpendek.
- Menggunakan BFS pada graf.
- Menguji beberapa pasangan kota.

```
hari-07-struktur-data > JS lab2.js > ...
267  function jarakTerpendek(
268    graf,
269    asal,
270    tujuan
271  ) {
272
273    const antrean = [
274      [asal, 0]
275    ];
276
277    const dikunjungi =
278      new Set([asal]);
279
280    while (antrean.length > 0) {
281
```

```
317  console.log(
318    '\nJarak Solo -> Jakarta'
319  );
320
321  console.log(
322    jarakTerpendek(
323      graf,
324      'Solo',
325      'Jakarta'
326    )
327  );
328
329  console.log(
330    '\nJarak Jogja -> Bandung'
331  );
332
333  console.log(
334    jarakTerpendek(
335      graf,
336      'Jogja',
337      'Bandung'
338    )
339  );
340
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab2.js
Jarak Solo -> Jakarta
3

Jarak Jogja -> Bandung
3
```

Lab 3 - Pencarian dan Pengurutan

1. Implementasi Algoritma Pencarian dan Pengurutan

- Membuat fungsi cariLinear().
- Membuat fungsi cariBiner().
- Membuat fungsi bubbleSort().
- Membuat fungsi mergeSort().
- Menguji array kosong, satu elemen, terurut, terbalik, dan duplikat.



```
hari-07-struktur-data > JS lab3.js > ...
function cariLinear(data, target) {

  for (let i = 0; i < data.length; i++) {

    if (data[i] === target) {
      return i;
    }

  }

  return -1;
}

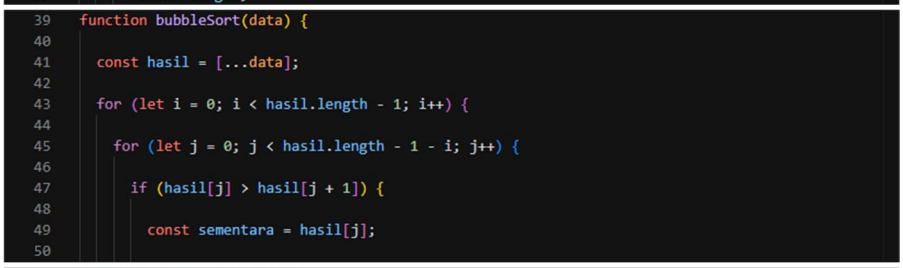
function cariBiner(data, target) {

  let kiri = 0;
  let kanan = data.length - 1;

  while (kiri <= kanan) {

    const tengah =
      Math.floor((kiri + kanan) / 2);

    if (data[tengah] === target) {
      return tengah;
```



```
function bubbleSort(data) {

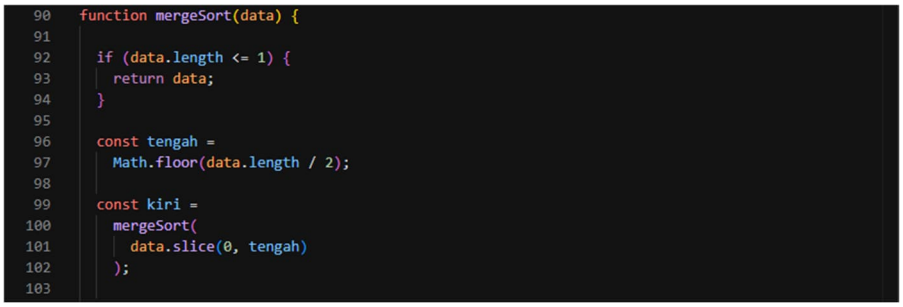
  const hasil = [...data];

  for (let i = 0; i < hasil.length - 1; i++) {

    for (let j = 0; j < hasil.length - 1 - i; j++) {

      if (hasil[j] > hasil[j + 1]) {

        const sementara = hasil[j];
```



```
function mergeSort(data) {

  if (data.length <= 1) {
    return data;
  }

  const tengah =
    Math.floor(data.length / 2);

  const kiri =
    mergeSort(
      data.slice(0, tengah)
    );
```



```
const satuElemen = [7];

const sudahTerurut = [
  1, 2, 3, 4, 5
];

const terbalik = [
  5, 4, 3, 2, 1
];

const duplikat = [
  5, 2, 5, 1, 2
];
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab3.js
Linear Search
2

Binary Search
2

Bubble Sort
[]
[ 7 ]
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 2, 5, 5 ]

Merge Sort
[]
[ 7 ]
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 2, 5, 5 ]
```

2. Perbandingan Bubble Sort dan Insertion Sort

- Membuat fungsi insertionSort().
- Menambahkan penghitung perbandingan.
- Menguji data acak.
- Menguji data terurut.
- Menguji data terbalik.

hari-07-struktur-data > JS lab3.js > ...

```
112  function bubbleSortHitung(data) {
113
114    const hasil = [...data];
115
116    let perbandingan = 0;
117
118    for (let i = 0; i < hasil.length - 1; i++) {
119
120      for (let j = 0; j < hasil.length - 1 - i; j++) {
121
122        perbandingan++;
123
124        if (hasil[j] > hasil[j + 1]) {
```

```
140  function insertionSortHitung(data) {
141
142    const hasil = [...data];
143
144    let perbandingan = 0;
145
146    for (let i = 1; i < hasil.length; i++) {
147
148      const sekarang = hasil[i];
149
150      let j = i - 1;
151
152      while (j >= 0) {
153
154        perbandingan++;
155
156        if (hasil[j] > sekarang) {
157
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab3.js
Perbandingan Data Acak
Bubble: 499500
Insertion: 254186

Perbandingan Data Terurut
Bubble: 499500
Insertion: 999

Perbandingan Data Terbalik
Bubble: 499500
Insertion: 499500
```

3. Waktu Eksekusi Algoritma

- Mengukur bubbleSort(), mergeSort(), dan Array.sort().
- Membandingkan hasil pengujian.
- Menganalisis hasil berdasarkan Big-O.

```
hari-07-struktur-data > JS lab3.js > ...
177  function buatDataAcak(jumlah) {
178
179    return Array.from(
180      { length: jumlah },
181      () => Math.floor(Math.random() * jumlah)
182    );
183
184  }
185
```

```
310  const data10k =
311    buatDataAcak(10000);
312
313  console.time('Bubble Sort 10K');
314
315  bubbleSort(data10k);
316
317  console.timeEnd('Bubble Sort 10K');
318
319  console.time('Merge Sort 10K');
320
321  mergeSort(data10k);
322
323  console.timeEnd('Merge Sort 10K');
```

```
331  const data100k =
332    buatDataAcak(100000);
333
334  console.time('Merge Sort 100K');
335
336  mergeSort(data100k);
337
338  console.timeEnd('Merge Sort 100K');
339
340  console.time('Array.sort 100K');
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node lab3.js
Bubble Sort 10K: 170.756ms
Merge Sort 10K: 22.076ms
Array.sort 10K: 4.015ms
Merge Sort 100K: 75.531ms
Array.sort 100K: 37.305ms
```

Operasi	Waktu (ms)	Big-O
Bubble Sort 10K	170.756	$O(n^2)$
Merge Sort 10K	22.076	O(n log n)
Array.sort 10K	4.015	O(n log n)
Merge Sort 100K	75.531	O(n log n)
Array.sort 100K	37.305	O(n log n)

Hasil pengujian menunjukkan bahwa Array.sort() memiliki waktu eksekusi paling cepat, diikuti oleh Merge Sort, sedangkan Bubble Sort paling lambat. Hal ini sesuai dengan teori Big-O karena Bubble Sort memiliki kompleksitas $O(n^2)$, sedangkan Merge Sort dan Array.sort() memiliki kompleksitas O(n log n). Perbedaan waktu semakin terlihat ketika jumlah data bertambah besar.

Lab 4 - Code Review dan Refactor

1. Analisis Kode Buruk

- Membuat file kode-buruk.js.
- Mengidentifikasi pelanggaran clean code.
- Mengisi tabel hasil analisis.



```
hari-07-struktur-data > JS kode-buruk.js > ...
1   var d = [80,65,90];
2
3   function f(a){
4     var s=0;
5     for(var i=0;i<a.length;i++){
6       s=s+a[i]
7     }
8     return s/a.length
9   }
10
11  function g(x){
12    if(x>=85){
13      return 'A'
14    }else{
15      if(x>=70){
16        return 'B'
17      }else{
18        if(x>=55){
19          return 'C'
20        }else{
21          return 'D'
22        }
23      }
24    }
25  }
26
27  var r=f(d);
28
29  if(r==80){
30    console.log('bagus')
```

No	Pelanggaran	Prinsip	Perbaikan
1	Nama d, f, g, r, h, k tidak bermakna	Penamaan	Gunakan nama yang deskriptif
2	Menggunakan var	Guideline modern	Gunakan const dan let
3	Format kode tidak rapi	Format	Rapikan dengan indentasi yang konsisten
4	Menggunakan ==	Perbandingan aman	Gunakan ===
5	Menggunakan angka ajaib (85, 70, 55, 0.11, 80)	Magic Number	Gunakan konstanta bernama
6	Fungsi h() dan k() identik	DRY	Gabungkan menjadi satu fungsi
7	Percabangan bertingkat pada g()	KISS	Gunakan return berurutan
8	Tidak menangani array kosong	Error Handling	Tambahkan pengecekan array kosong
9	Titik koma tidak konsisten	Konsistensi	Gunakan format yang konsisten
10	Tidak ada komentar atau JSDoc	Dokumentasi	Tambahkan dokumentasi fungsi

2. Refactor Kode

- Membuat file kode-bersih.js.
- Menerapkan perbaikan sesuai hasil analisis.
- Menguji hasil refactor.



```
hari-07-struktur-data > JS kode-baik.js > ...
1   const PAJAK = 0.11;
2
3   const BATAS_GRADE = {
4     A: 85,
5     B: 70,
6     C: 55
7   };
8
9   /**
10   * Menghitung rata-rata nilai.
11   */
12  function hitungRataRata(daftarNilai) {
13
14    if (daftarNilai.length === 0) {
15      return 0;
16    }
17
18    const jumlah =
19      daftarNilai.reduce(
20        (total, nilai) => total + nilai,
21        0
22      );
23
24    return jumlah / daftarNilai.length;
25  }
26
```



```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node kode-buruk.js
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> node kode-baik.js
B
```

Lab 5 - ESLint dan Prettier

1. Instalasi ESLint dan Prettier

- Membuat konfigurasi proyek Node.js.
- Menginstal ESLint dan Prettier.
- Menginstal Prettier.
- Menyiapkan file konfigurasi.

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> npm init -y
Wrote to E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data\package.json:

{
  "name": "hari-07-struktur-data",
  "version": "1.0.0",
  "description": "",
  "main": "kode-baik.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-07-struktur-data> npm install --save-dev eslint @eslint/js prettier


added 79 packages, and audited 80 packages in 8s

18 packages are looking for funding
  run `npm fund` for details

5 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

2. Konfigurasi ESLint

- Membuat file eslint.config.js.
- Menambahkan aturan ESLint.

```
hari-07-struktur-data > eslint.config.js > ...
1   import js from '@eslint/js';
2
3   export default [
4     js.configs.recommended,
5     {
6       languageOptions: {
7         ecmaVersion: 2022,
8         sourceType: 'module',
9         globals: {
10          document: 'readonly',
11          window: 'readonly',
12          console: 'readonly',
13        },
14      },
15      rules: {
16        eqeqeq: 'error',
17        'no-var': 'error',
18        'prefer-const': 'warn',
19        'no-unused-vars': 'warn',
20      },
21    },
22  ];
23
```

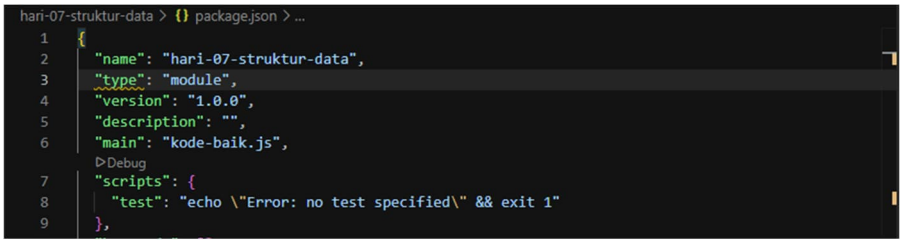
3. Konfigurasi Prettier

- Membuat file .prettierrc.
- Menentukan aturan format kode.



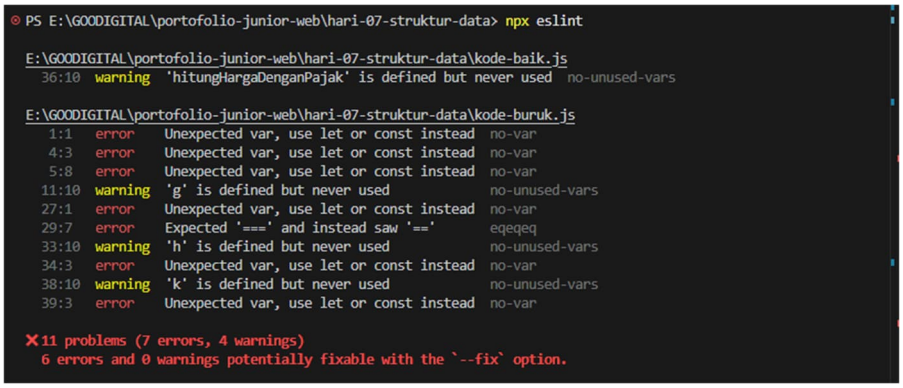
4. Type Module

- Mengubah konfigurasi package.json.
- Mengaktifkan ES Module.



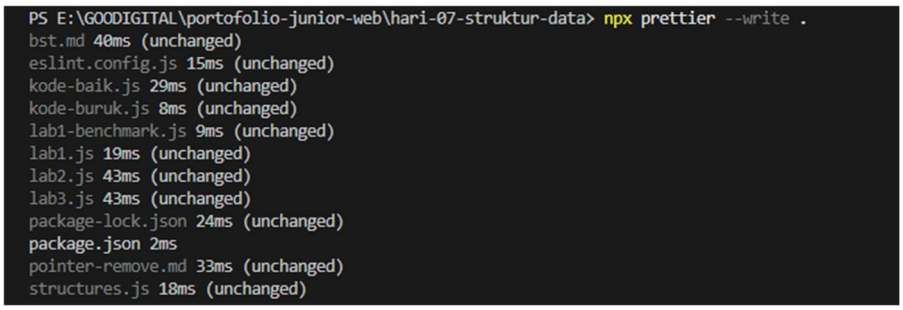
5. Menjalankan ESLint

- Menjalankan pemeriksaan kode.
- Mengidentifikasi error dan warning.



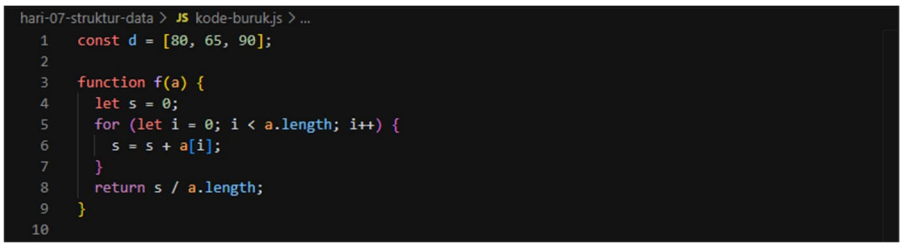
6. Menjalankan Prettier

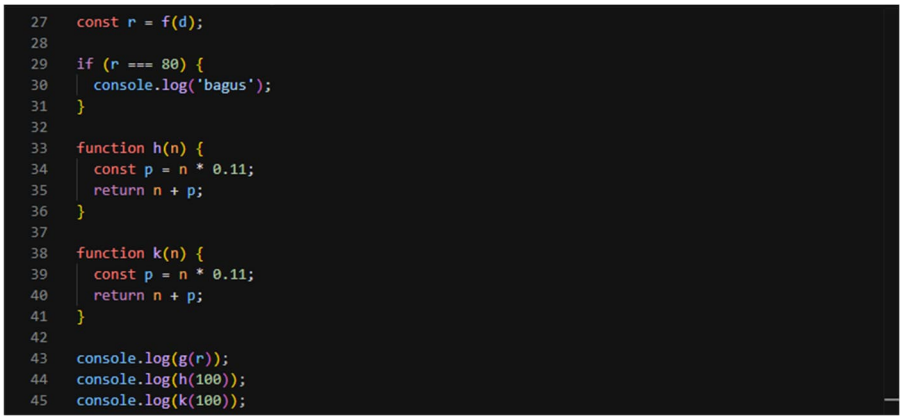
- Memformat kode secara otomatis.
- Merapikan indentasi dan spasi.



7. Memperbaiki Error ESLint

- Memperbaiki penggunaan var.
- Mengganti == menjadi ===.
- Mengurangi warning yang tidak diperlukan.
- Menjalankan ulang ESLint.





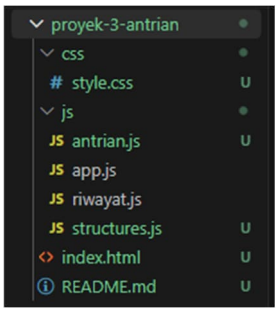


Kondisi	Error	Warning
Sebelum Perbaikan	7	4
Sesudah Perbaikan	0	0

PBL Hari 7 - Proyek 3: Simulator Antrean dan Riwayat

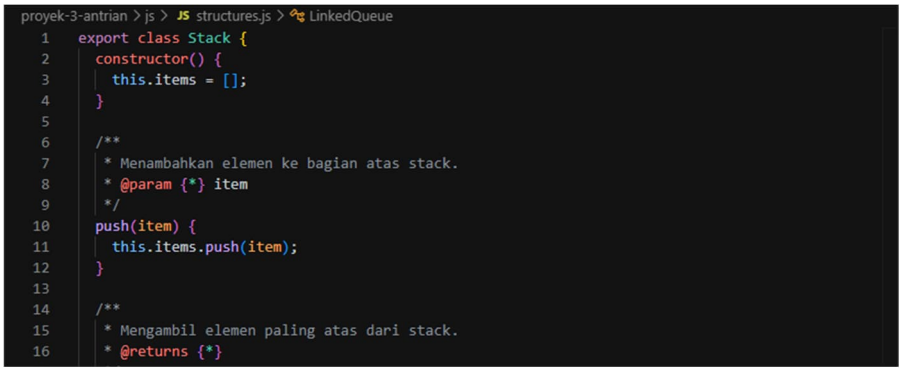
1. Menyiapkan Struktur Proyek

- Membuat folder: proyek-3-antrian, css, dan js.
- Membuat file: index.html, style.css, structures.js, antrian.js, riwayat.js, dan app.js.
- Membuat file README.md.



2. Membuat Structure Data Proyek

- Membuat class Stack, class Node, class LinkedList, dan class LinkedQueue.
- Menambahkan komentar fungsi.







3. Membuat Antarmuka Proyek

- Membuat file index.html.
- Membuat area antrean loket.
- Membuat area editor catatan.
- Membuat file style.css.
- Menyiapkan pemanggilan app.js.

proyek-3-antrian > <> index.html > ... Text Editor

```html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Sistem Antrean dan Catatan Klinik</title>

  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <h1>Sistem Antrean dan Catatan Klinik</h1>

  <section>

    <h2>Antrean Loket</h2>

    <p>
      Sedang dipanggil:
      <strong id="dipanggil">-</strong>
    </p>

    <p>
      Menunggu:
      <span id="jumlah">0</span>
    </p>
```

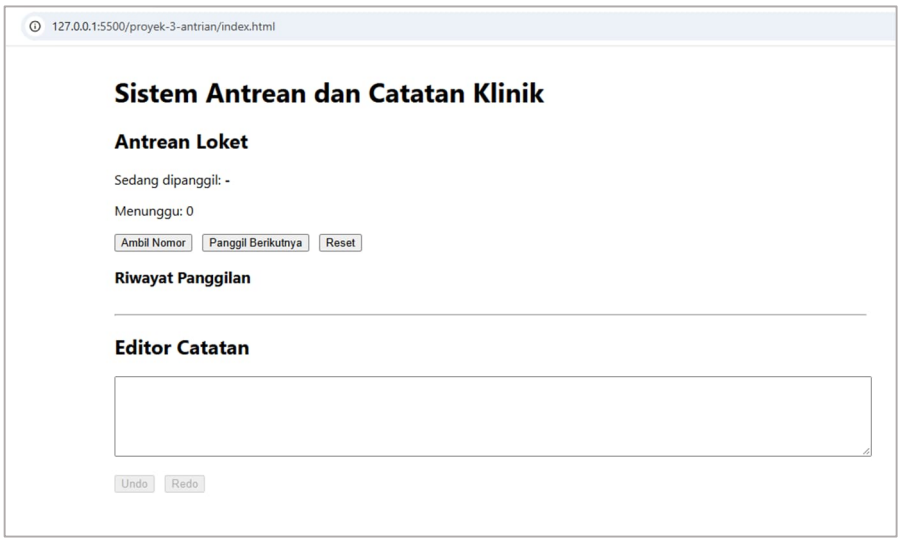
proyek-3-antrian > css > # style.css > ...

```css
body {
  font-family: system-ui, sans-serif;
  max-width: 900px;
  margin: 2rem auto;
}

section {
  margin-bottom: 2rem;
}

textarea {
  width: 100%;
}

button {
  margin-right: 0.5rem;
}
```



4. Membuat Logika Antrean Loket

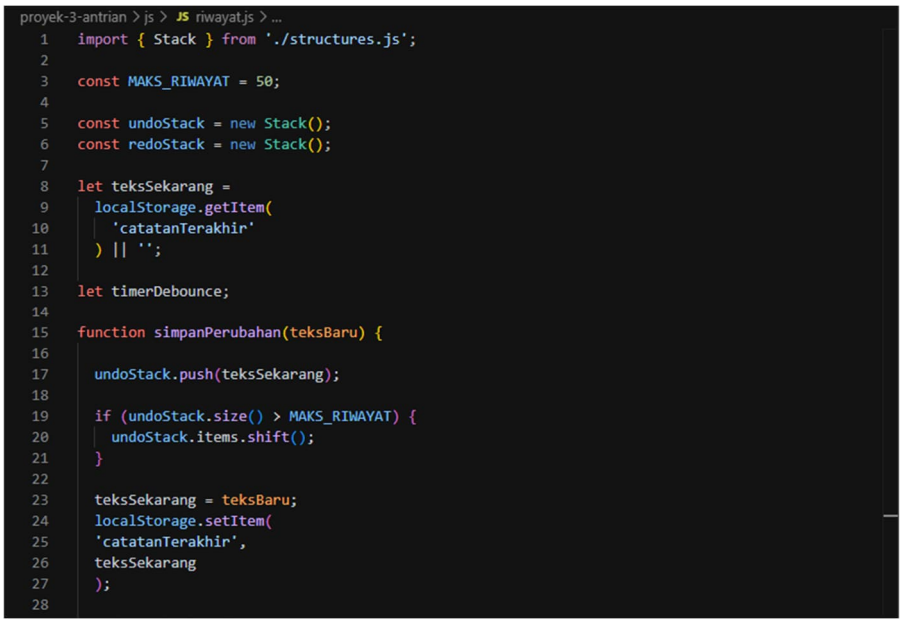
- Membuat file antrian.js.
- Menggunakan LinkedQueue.
- Membuat fungsi ambil nomor antrean.
- Membuat fungsi panggil nomor antrean.



```
proyek-3-antrian > js > JS antrian.js > ...
1  import { LinkedQueue } from './structures.js';
2
3  const AWALAN_NOMOR = 'A';
4
5  const antrian = new LinkedQueue();
6
7  let nomorTerakhir = 0;
8
9  function formatNomor(nomor) {
10   return AWALAN_NOMOR +
11     String(nomor).padStart(3, '0');
12 }
13
14 function ambilNomor() {
15
16   nomorTerakhir++;
17
18   const nomor =
19     formatNomor(nomorTerakhir);
20
21   antrian.enqueue(nomor);
22
23   return nomor;
24 }
```

5. Membuat Logika Riwayat Undo/Redo

- Membuat file riwayat.js.
- Menggunakan dua Stack untuk Undo dan Redo.
- Membatasi riwayat maksimal 50 langkah.
- Menambahkan debounce saat mengetik.
- Menyimpan catatan terakhir.

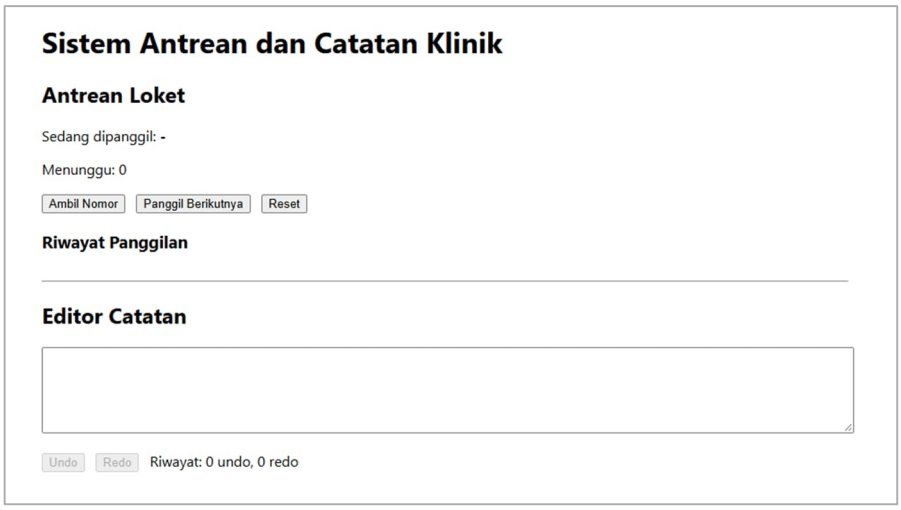


```
proyek-3-antrian > js > JS riwayat.js > ...
1  import { Stack } from './structures.js';
2
3  const MAKS_RIWAYAT = 50;
4
5  const undoStack = new Stack();
6  const redoStack = new Stack();
7
8  let teksSekarang =
9    localStorage.getItem(
10     'catatanTerakhir'
11   ) || '';
12
13 let timerDebounce;
14
15 function simpanPerubahan(teksBaru) {
16
17   undoStack.push(teksSekarang);
18
19   if (undoStack.size() > MAKS_RIWAYAT) {
20     undoStack.items.shift();
21   }
22
23   teksSekarang = teksBaru;
24   localStorage.setItem(
25   'catatanTerakhir',
26   teksSekarang
27   );
28
```

6. Membuat Penghubung DOM

- Membuat file app.js.
- Menghubungkan antarmuka dengan antrian.js.
- Menghubungkan antarmuka dengan riwayat.js.
- Menampilkan data antrean dan riwayat editor pada halaman.

```
proyek-3-antrian > js > JS app.js > ...
1   import {
2     antrian,
3     ambilNomor,
4     panggilNomor
5   } from './antrian.js';
6
7   import {
8     simpanDenganDebounce,
9     undo,
10    redo,
11    infoRiwayat
12  } from './riwayat.js';
13
14  const editor = document.getElementById('editor');
15
16  const tombolUndo =
17    document.getElementById('undo');
18
19  const tombolRedo =
20    document.getElementById('redo');
21
22  const info =
23    document.getElementById('info');
24
25  const dipanggil =
26    document.getElementById('dipanggil');
27
28  const jumlah =
29    document.getElementById('jumlah');
30
```



7. Menguji Simulator Antrean dan Riwayat

- Menguji pengambilan nomor antrean.
- Menguji pemanggilan nomor secara FIFO.
- Menguji riwayat 5 nomor terakhir yang dipanggil.
- Menguji tombol Reset antrean.
- Menguji Undo dan Redo pada editor catatan.

Sistem Antrean dan Catatan Klinik

Antrean Loket

Sedang dipanggil: **-**

Menunggu: 5

Ambil Nomor | Panggil Berikutnya | Reset

1. A001
2. A002
3. A003
4. A004
5. A005

Riwayat Panggilan

Sistem Antrean dan Catatan Klinik

Antrean Loket

Sedang dipanggil: **A002**

Menunggu: 3

Ambil Nomor | Panggil Berikutnya | Reset

1. A003
2. A004
3. A005

Riwayat Panggilan

1. A002
2. A001

Sistem Antrean dan Catatan Klinik

Antrean Loket

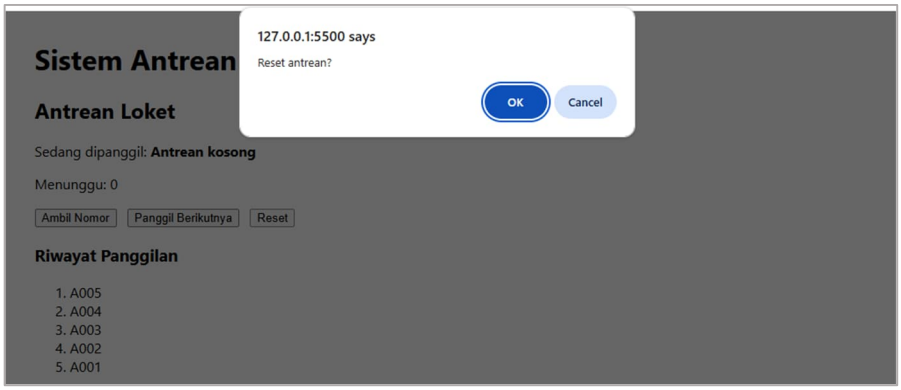
Sedang dipanggil: **Antrean kosong**

Menunggu: 0

Ambil Nomor | Panggil Berikutnya | Reset

Riwayat Panggilan

1. A005
2. A004
3. A003
4. A002
5. A001



Sistem Antrean dan Catatan Klinik

Antrean Loket

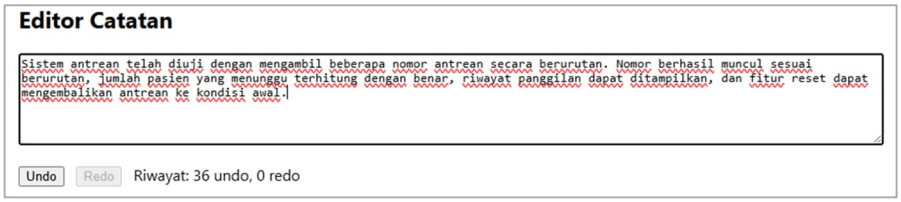
Sedang dipanggil: **-**

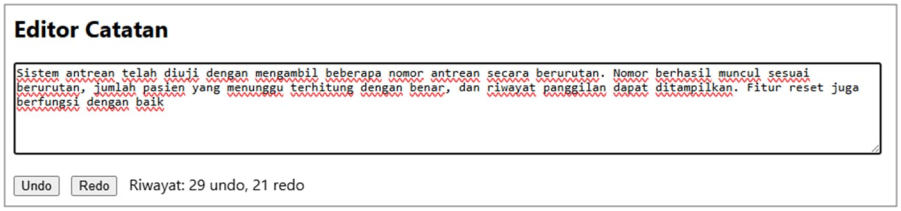
Menunggu: 3

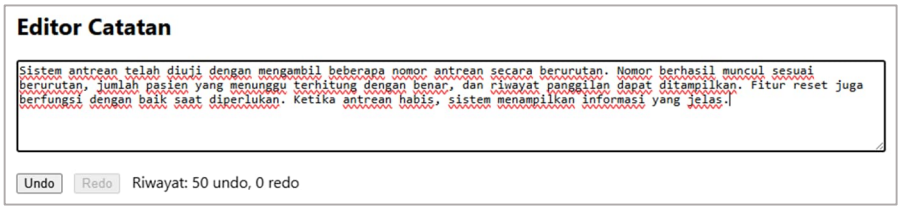
Ambil Nomor | Panggil Berikutnya | Reset

1. A006
2. A007
3. A008

Riwayat Panggilan

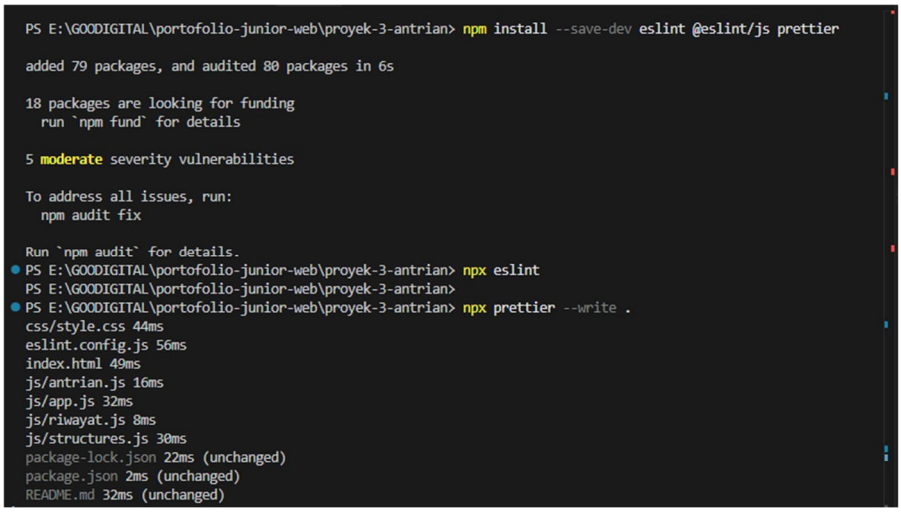






8. Menjalankan ESLint dan Prettier

- Menginstal ESLint dan Prettier pada proyek.
- Menjalankan pemeriksaan kode menggunakan ESLint.
- Menjalankan format otomatis menggunakan Prettier.



Pemeriksaan	Hasil
Error	0
Warning	0
Status	Lulus pemeriksaan ESLint

Tools	Hasil
ESLint	Tidak ditemukan error maupun warning
Prettier	Kode berhasil diformat sesuai konfigurasi

9. README

- Membuat file README.md.
- Menambahkan cara menjalankan, tangkapan layar, dan tabel alasan pemilihan struktur data.



10. Review

Butir	Ya/Tidak	Catatan
Penamaan deskriptif dan konsisten	☑ Ya	Nama variabel dan fungsi sudah cukup jelas sehingga mudah dipahami saat dibaca ulang.
Indentasi dan format seragam (Prettier)	☑ Ya	Format kode sudah rapi dan konsisten setelah dijalankan dengan Prettier.
Fungsi pendek, satu tanggung jawab	☑ Ya	Sebagian besar fungsi hanya menangani satu pekerjaan sehingga lebih mudah dipelihara.
Tidak ada duplikasi dan kode mati	☑ Ya	Kode yang tidak digunakan sudah dibersihkan dan fungsi yang sama tidak ditulis berulang.
Tidak ada magic number	☑ Ya	Sudah menggunakan konstanta seperti `MAKS_RIWAYAT` dan `AWALAN_NOMOR`.
Error ditangani dengan pesan jelas	☑ Ya	Kondisi antrean kosong dan struktur data kosong sudah ditangani dengan pesan yang mudah dipahami.
Struktur data tidak mengakses DOM	☑ Ya	`structures.js` hanya berisi struktur data, sedangkan DOM ditangani pada file lain.
Lolos ESLint tanpa error	☑ Ya	Hasil pemeriksaan terakhir tidak menunjukkan error maupun warning.

11. Repository GitHub

- Melakukan commit.
- Melakukan push ke GitHub.
- Tautan repository: https://github.com/amribaros15/portofolio-junior-web/

